

Programação – Exame da Época Normal

23 de junho de 2025 – Duração: 120 minutos

LEI, LEI-PL, LEI-CE

1. Considere as seguintes definições:

```
typedef struct dados item;  
  
struct dados{  
    int id;                // Identificador numérico único  
    char nome[100];        // Nome do item  
    int qtd;               // Quantidade em stock  
    int minstock;          // Quantidade mínima para gerar alerta de stock  
};
```

Uma pastelaria armazena num **ficheiro binário** informação sobre os ingredientes que mantém em stock. É um ficheiro constituído exclusivamente por estruturas do tipo **item**. Cada estrutura armazena informação sobre um ingrediente: **id numérico único, nome do ingrediente, quantidade em stock e valor limite para gerar alerta de falta de stock**. As estruturas no ficheiro estão ordenadas pelo valor do *id*. Pode assumir que o identificador do primeiro ingrediente é o 1 e que os seguintes surgem sequencialmente, sem nenhum id em falta. Ao lado, pode consultar um exemplo de um ficheiro binário com 4 ingredientes (ids 1 a 4).

As receitas da pastelaria estão armazenadas em ficheiros de texto. Cada ficheiro contém toda a informação relativa aos ingredientes necessários para confeccionar um bolo. O ficheiro de texto está organizado em linhas, cada linha indicando um dos ingredientes necessários para essa receita, no seguinte formato:

Id : Nome : Quantidade : Comentário

A seguir pode consultar um exemplo de um ficheiro de texto para uma receita com 3 ingredientes:

Ing.bin

1 Farinha 1200 200
2 Leite Magro 3000 500
3 Ovos XXL 100 12
4 Sal 505 10

2 : Leite Magro : 100 : O leite deve estar morno

3 : Ovos XXL : 6 : Nada a referir

1 : Farinha : 250 : Farinha T55

Escreva uma função em C que atualize a quantidade em stock dos ingredientes que vão ser usados numa receita. Pode assumir que já foi efetuada uma validação prévia: a receita está correta, os ingredientes existem e têm stock suficiente para o que vai ser confeccionado. **A função deve atualizar a quantidade em stock** de cada um dos ingredientes que vai ser usado na receita (retirando a quantidade indicada na receita). No exemplo em cima serão usados **6 Ovos XXL**, logo a **quantidade em stock no ficheiro binário passa a 94**.

A função deve igualmente gerar **alertas para stocks baixos**. Se, depois da atualização, o stock que resta de um ingrediente for **inferior à quantidade mínima**, deve ser escrito na consola: **O ingrediente Nome com ID Id tem stock abaixo do mínimo**.

A função recebe os nomes de **2 ficheiros como parâmetro**: o ficheiro binário onde está armazenada informação sobre os stocks dos ingredientes e o ficheiro de texto com a receita. **Devolve 1** se tudo correr bem, **ou 0**, no caso de existir algum problema no acesso aos ficheiros.

[Cotação: 30%]

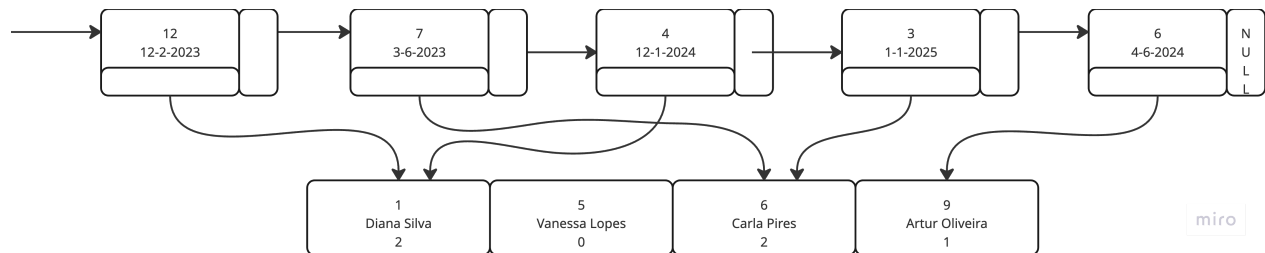
Considere as seguintes definições:

```
typedef struct projeto proj, *pproj;
typedef struct pessoa gestor, *pgest;

struct pessoa{
    int id; // Identificador único do gestor
    char nome[100]; // Nome do gestor
    int total; // Número de projetos que está a gerir
};

struct projeto{
    int id; // Identificador único do projeto
    struct {int dia, mes, ano;} inicio; // Data de início do projeto
    pgest p; // Ponteiro para o gestor deste projeto
    pproj prox;
};
```

Uma empresa de construção armazena numa estrutura dinâmica informações sobre os seus projetos em curso, associando a cada projeto a identificação do seu gestor. A estrutura contém uma lista ligada simples constituída por nós do tipo *proj*, um nó por cada projeto em curso (caracterizado pelo id único do projeto e pela data de início). A lista não está ordenada por nenhum critério em particular e não existem 2 projetos que tenham começado no mesmo dia. Existe ainda 1 vetor dinâmico, com estruturas do tipo *gestor*, cada uma delas contendo informação sobre um funcionário da empresa que é gestor de projeto. O exemplo da figura ilustra uma situação em que existem 5 projetos e 4 gestores. A partir de cada projeto armazenado na lista, o campo *p* permite aceder ao seu gestor (o gestor com id 6 está a gerir os projetos com id 3 e 7). O vetor dinâmico está **ordenado pelo valor do campo *id***.



2. Escreva uma função em C que descubra quando se iniciaram o projeto mais antigo e o projeto mais recente de um gestor. A função recebe, como parâmetros, o identificador do gestor em causa, o endereço inicial da lista, o endereço inicial do vetor e a dimensão do vetor. Escreve na consola as datas de início dos projetos. Caso o gestor não exista ou tenha menos de 2 projetos em curso, a função escreve na consola “Não aplicável”.

[Cotação: 25%]

3. Escreva uma função em C que elimine um gestor do vetor dinâmico. Os seus projetos devem ser transferidos para o gestor que se mantém na empresa e que tenha menos projetos em curso (pode definir o critério de desempate que preferir). Se o vetor original só tiver um gestor ou se o gestor a eliminar não existir, a função não executa nenhuma modificação.

A função recebe, como parâmetros, o identificador do gestor a eliminar, o endereço inicial da lista, o endereço inicial do vetor e o endereço de uma variável inteira onde se encontra a dimensão do vetor. Devolve o endereço do vetor depois da atualização. Se existir algum problema de gestão dinâmica, a função devolve o vetor original (sem nenhuma alteração nos dados da estrutura dinâmica).

[Cotação: 35%]

Programação – Exame da Época Normal

23 de junho de 2025 – Duração: 120 minutos

LEI, LEI-PL, LEI-CE

Nome: _____ Número: _____

4. Funções Recursivas

Qual é o output na consola quando é executado o seguinte programa em C?

```
#include <stdio.h>
#define TAM 6

void f(int a[], int n){
    if(n <= 1)
        return;
    printf("%d ", *a);
    f(a+1, n-2);
    printf("%d ", a[n-1]);
}

int main(void) {
    int a[TAM] = {1, 2, 3, 4, 5, 6};

    f(a, TAM);
    return 0;
}
```

Resposta:

☐ 1 2 3 3 2 1

☐ 3 2 1 1 2 3

☐ 1 2 3 4 5 6

☐ 1 2 3 6 5 4

☐ 3 2 1 4 5 6

☐ 3 2 1 6 5 4

[Cotação: 10%] (uma resposta errada não desconta)